



Computer Science Engineering
Software Engineering Lab

A. Y. 2025-26

Name of Student	Atharva Pramod Hule
UID	2024800038
Batch	A3
Class / Branch	T.E. Computer Science & Engineering
Experiment No.	3
Title	Design Class Diagram – Green Finance (Carbon Credit & Fintech Platform)

Aim: Design Class Diagram for the Green Finance Project

Objective:

- To study and understand the principles of Object-Oriented Analysis and Design (OOAD). • To identify classes, attributes, and methods for the Green Finance carbon-credit and fintech platform.
- To determine relationships (association, aggregation, composition, inheritance) between classes.
- To represent the static structure of the Green Finance system using a UML Class Diagram. • To prepare a visual blueprint that can guide the object-oriented implementation of the backend.

Project Description:

Green Finance is a fintech platform designed to manage carbon credits as a tradeable, account-linked asset. The system tracks a user's carbon emissions, converts reduction in emissions into carbon credits, and allows those credits to be transferred, retired, or settled through a proper double-entry ledger — similar to how a bank manages transactions. The backend is being built using **FastAPI**, following a layered architecture (API → Service → Model → Database), and is planned to evolve from a Phase 1 monolith into a Phase 2 microservices architecture with an ML-based rebalancing engine.

The class diagram below models the core domain entities of this system — **User**, **Account**, **CarbonRecord**, **CarbonCredit**, **Credit**, **Transaction**, and **LedgerEntry** — and defines how they are related to one another.

Class Diagram:

[Paste / Insert the Green Finance UML Class Diagram image here]

(Recommended: export the draw.io diagram as PNG at 300 DPI before inserting)

Classes Identified:

User	Stores identity, login credentials, and profile of a platform user.
Account	Holds the user's financial balance linked to carbon-credit value.
CarbonRecord	Stores emission activity data used to calculate carbon footprint.
CarbonCredit	Represents a carbon credit earned, issued, or transferred by a user.
Credit	Represents a retirement record when a carbon credit is used up.
Transaction	Represents any financial or credit-related action performed by a user.
LedgerEntry	Stores debit/credit accounting entries for each transaction.

Relationships Identified:

User → Account	Composition	1 : 1	An account cannot exist without its owning user; it is deleted along with the user.
User → CarbonRecord	Association	1 : *	A user can log many emission activities over time.
User → CarbonCredit	Association	1 : *	A user can earn many carbon credits independently over time.
User → Transaction	Association	1 : *	A user can initiate many transactions.
CarbonCredit → Credit	Composition	1 : *	A retirement record has no meaning without its parent carbon credit.
Transaction → LedgerEntry	Composition	1 : 2..*	Every transaction must create at least a debit and a credit ledger entry; entries cannot exist without the transaction.

Account → LedgerEntry	Association	1 : *	An account is referenced by many ledger entries that affect its balance.
------------------------------	-------------	-------	--

Procedure:

- **Step 1 – Identify Classes:** Studied the Green Finance backend requirements and identified the core entities – User, Account, CarbonRecord, CarbonCredit, Credit, Transaction, and LedgerEntry.
- **Step 2 – List Attributes and Methods:** For each class, attributes (with data types) and methods (with visibility) were listed based on the planned FastAPI models and services.
- **Step 3 – Identify Relationships:** Determined how classes relate – which relationships are strong ownership (composition) and which are simple references (association) – along with their multiplicity.
- **Step 4 – Create Class Boxes:** Represented each class as a UML class box divided into name, attribute, and method compartments.
- **Step 5 – Add Attributes and Methods:** Filled each compartment using visibility notations (+ public, - private).
- **Step 6 – Draw Relationships:** Connected the class boxes using UML notations – filled diamond for composition, plain arrow for association.
- **Step 7 – Label Relationships:** Added relationship names and multiplicities (e.g., 1, *, 2..*) next to each connector for clarity.
- **Step 8 – Review and Refine:** Reviewed the diagram against the actual backend model files to ensure it accurately reflects the Green Finance data structure.

Conclusion:

Through this experiment, I learned how to design a UML class diagram for the Green Finance project by identifying its core classes, attributes, methods, and the relationships between them. I understood the difference between composition (a strong, ownership-based relationship where the part cannot exist without the whole, such as User–Account) and association (a simple reference relationship, such as User–Transaction). This exercise helped me visualize the static structure of the Green Finance backend before implementation, which will directly guide how the SQLAlchemy models and FastAPI services are structured in the actual codebase.

In conclusion, this class diagram serves as the blueprint for the Green Finance data layer, connecting user identity, carbon tracking, credit management, and financial ledgering into one coherent object-oriented design.